

1) Dado el siguiente dataset:

Split 1		Split 2		Split 3		Split 4	
Key	Value	Key	Value	Key	Value	Key	Value
34	21	23	45	3	21	30	91
21	34	12	12	15	10	31	32
10	18	36	18	14	18	32	53
32	45	4	97	3	15	19	35

a) Job A

```
def map(k1, v1, context):  
    context.write(1, v1)  
  
def reduce(k2, v2, context):  
    n = 0  
    for v in v2:  
        n = n + 1  
    context.write(k2, n)
```

Responda para cada job:

- ¿Cuántas veces (invocaciones) se ejecuta la función map?
- ¿Cuántas veces (invocaciones) se ejecuta la función reduce?
- ¿Cuántos mappers se ejecutan?
- ¿Cuántos reducers se ejecutan?
- ¿Qué datos recibe cada función reduce?
- ¿Cuál es la salida de cada job?

Responda para cada job:

- ¿Cuántas veces (invocaciones) se ejecuta la función map?
La función map se va a ejecutar 1 vez por cada par de cada Split, en el Split1 tenemos 4 pares, en el Split2 4... $4 \times 4 = 16$ veces se va a ejecutar el map.. una por cada fila (par [key value])
- ¿Cuántas veces (invocaciones) se ejecuta la función reduce?
En MapReduce, el **reduce** se ejecuta **una vez por cada key distinta** que sale del **map**. Como solamente hay **una key distinta: 1**, se ejecuta una sola vez el reduce

- **map** → una vez por cada registro de entrada → 16
- **reduce** → una vez por cada key distinta que genera map → 1

- ¿Cuántos mappers se ejecutan?
Una x cada Split... 4
- ¿Cuántos reducers se ejecutan?
- ¿Qué datos recibe cada función reduce?
- ¿Cuál es la salida de cada job?

.....

a) Job A

- ¿Cuántas veces se ejecuta la función map? 16 (Una por cada línea)
- ¿Cuántas veces se ejecuta la función reduce? 4 (Una por cada split)
- ¿Cuántos mappers se ejecutan? Se ejecutan 4 mappers (uno por cada nodo en el mejor de los casos)
- ¿Cuántos reducers se ejecutan? Se ejecuta 1 reducer ya que al escribir (1, v1) se agrupan todos los valores en una sola clave (por lo que no permite la concurrencia)
- ¿Qué datos recibe cada función reduce? Recibe una lista de valores

Función map(k1, v1)

```
# Genera un solo archivo con todos Los valores
# por ejemplo
# (1, "21")
# (1, "34")
# (1, "18")
def map(k1, v1, context):
    context.write(1, v1)
```

Función reduce(k2, v2)

```
# Recibe una Lista con todos Los valores
# Y cuenta Los valores simplemente
# Retorna k2 que es 1 y n que seria La cantidad
# de valores en este caso 16
def reduce(k2, v2, context):
    n = 0
    for v in v2:
        n = n + 1
    context.write(k2, n)
```

Split 1		Split 2		Split 3		Split 4	
Key	Value	Key	Value	Key	Value	Key	Value
34	21	23	45	3	21	30	91
21	34	12	12	15	10	31	32
10	18	36	18	14	18	32	53
32	45	4	97	3	15	19	35

¿Cuál es la salida de cada job? La salida de cada Job es (1, 16)

=====

b) Job B

- ¿Cuántas veces se ejecuta la función map? 16 (Una por cada key diferente)
- ¿Cuántas veces se ejecuta la función reduce? 4 (Una por cada split)
- ¿Cuántos mappers se ejecutan? Se ejecutan 4 mappers (uno por cada nodo en el mejor de los casos)
- ¿Cuántos reducers se ejecutan? Se ejecuta 1 reducer ya que al escribir (1, v1) se agrupan todos los valores en una sola clave (por lo que no permite la concurrencia)
- ¿Qué datos recibe cada función reduce? Recibe una lista de valores

Función map(k1, v1)

```
# Genera un solo archivo con todos Los valores
# por ejemplo
# (1, "21")
# (1, "34")
# (1, "18")
def map(k1, v1, context):
    context.write(1, v1)
```

Función reduce(k2, v2)

```
# Recibe una lista con todos Los valores
# Y suma Los valores simplemente
def reduce(k2, v2, context):
    n = 0
    for v in v2:
        n = n + v
    context.write(k2, n)
```

Split 1		Split 2		Split 3		Split 4	
Key	Value	Key	Value	Key	Value	Key	Value
34	21	23	45	3	21	30	91
21	34	12	12	15	10	31	32
10	18	36	18	14	18	32	53
32	45	4	97	3	15	19	35

¿Cuál es la salida de cada job? La salida de cada Job es (1, 565)

- Split 1 -> 21 + 34 + 18 + 45 = 118
- Split 2 -> 45 + 12 + 18 + 97 = 172
- Split 3 -> 21 + 10 + 18 + 15 = 64
- Split 4 -> 91 + 32 + 53 + 35 = 211
- Total -> 118 + 172 + 64 + 211 = 565

Split 1		Split 2		Split 3		Split 4	
Key	Value	Key	Value	Key	Value	Key	Value
34	21	23	45	3	21	30	91
21	34	12	12	15	10	31	32
10	18	36	18	14	18	32	53
32	45	4	97	3	15	19	35

b) Job B

```
def map(k1, v1, context):
    context.write(1, v1)

def reduce(k2, v2, context):
    n = 0
    for v in v2:
        n = n + v
    context.write(k2, n)
```

- ¿Cuántas veces (invocaciones) se ejecuta la función map?
- ¿Cuántas veces (invocaciones) se ejecuta la función reduce?
- ¿Cuántos mappers se ejecutan?
- ¿Cuántos reducers se ejecutan?
- ¿Qué datos recibe cada función reduce?
- ¿Cuál es la salida de cada job?

16

4

4 si hay nodos y recursos disponibles

1 solo, ya que la clave siempre es 1

va recibir 1 y la sumatorio del campo Value de ese Split

Como va ser un solo Reduce va terminar siendo algo como 1, Sumatoria Value(all splits)

1) Dado el siguiente dataset:

Split 1		Split 2		Split 3		Split 4	
Key	Value	Key	Value	Key	Value	Key	Value
34	21	23	45	3	21	30	91
21	34	12	12	15	10	31	32
10	18	36	18	14	18	32	53
32	45	4	97	3	15	19	35

c) Job C

```
def map(k1, v1, context):
    if (v1 < 30):
        context.write(1, k1)
    else:
        context.write(2, k1)

def reduce(k2, v2, context):
    max = -1
    for v in v2:
        if (v > max):
            max = v
    context.write(k2, max)
```

- ¿Cuántas veces (invocaciones) se ejecuta la función map?
- ¿Cuántas veces (invocaciones) se ejecuta la función reduce?
- ¿Cuántos mappers se ejecutan?
- ¿Cuántos reducers se ejecutan?
- ¿Qué datos recibe cada función reduce?
- ¿Cuál es la salida de cada job?

16 una por cada fila de cada Split
8 por cada split
4 mappers 1 x split
2.. 1 por cada clave intermedia
recibe una lista de keys

Primero se filtra por los valores menores a 30 y guarda las keys

¿Cuál es la salida de cada job?

(1, [34, 10, 12, 36, 3, 15, 14, 3]) -> De este se queda el mayor 36
(2, [21, 32, 23, 4, 30, 31, 32, 19]) -> De este se queda el mayor 32

El resultado es (1, 36) (2, 32)

Aca preguntar...

cuando tenemos ifs, la cantidad de reduces que se ejecutan es la cantidad de context.write(valorDiferente, k1)?

⇒ CLAVE

~~o si o si hay que analizar si por lo menos entro 1 vez en el if y else para saber que tuvimos 2 reduces que se ejecutaron~~

1) Dado el siguiente dataset:

Split 1		Split 2		Split 3		Split 4	
Key	Value	Key	Value	Key	Value	Key	Value
<u>34</u>	21	<u>23</u>	45	<u>3</u>	21	<u>30</u>	91
<u>21</u>	34	<u>12</u>	12	<u>15</u>	10	<u>31</u>	32
<u>10</u>	18	<u>36</u>	18	<u>14</u>	18	32	53
32	45	<u>4</u>	97	<u>3</u>	15	<u>19</u>	35

- ¿Cuántas veces (invocaciones) se ejecuta la función map?
- ¿Cuántas veces (invocaciones) se ejecuta la función reduce?
- ¿Cuántos mappers se ejecutan?
- ¿Cuántos reducers se ejecutan?
- ¿Qué datos recibe cada función reduce?
- ¿Cuál es la salida de cada job?

d) Job D

```
def map(k1, v1, context):
    for v in range(v1):
        context.write(k1, v1)

def reduce(k2, v2, context):
    n = 0
    for v in v2:
        n = n + 1
    context.write(k2, n)
```

16 invocaciones.. 1 x fila de split
 14 se ejecuta 1x cada clave unica <>
 4 mappers
 14 reduces x que tiene 14 claves unicas <>
 Cada reducer recibe una lista de Value elementos, todos iguales, generados por el map.

¿Cuál es la salida de cada job?

```
(34, 21), (34, 21), ... (21 veces)
(21, 34), (21, 34), ... (34 veces)
...
(19, 35), (19, 35), ... (35 veces)
```

Las salidas del job son (Key, cantidad) donde cantidad es el número de repeticiones, equivalente a Value.

1) Dado el siguiente dataset:

Split 1		Split 2		Split 3		Split 4	
Key	Value	Key	Value	Key	Value	Key	Value
34	<u>21</u>	23	45	3	21	30	<u>91</u>
21	<u>34</u>	12	<u>12</u>	15	<u>10</u>	31	<u>32</u>
10	18	36	<u>18</u>	14	18	32	<u>53</u>
32	<u>45</u>	4	<u>97</u>	3	<u>15</u>	19	<u>35</u>

- ¿Cuántas veces (invocaciones) se ejecuta la función map?
- ¿Cuántas veces (invocaciones) se ejecuta la función reduce?
- ¿Cuántos mappers se ejecutan?
- ¿Cuántos reducers se ejecutan?
- ¿Qué datos recibe cada función reduce?
- ¿Cuál es la salida de cada job?

e) Job E

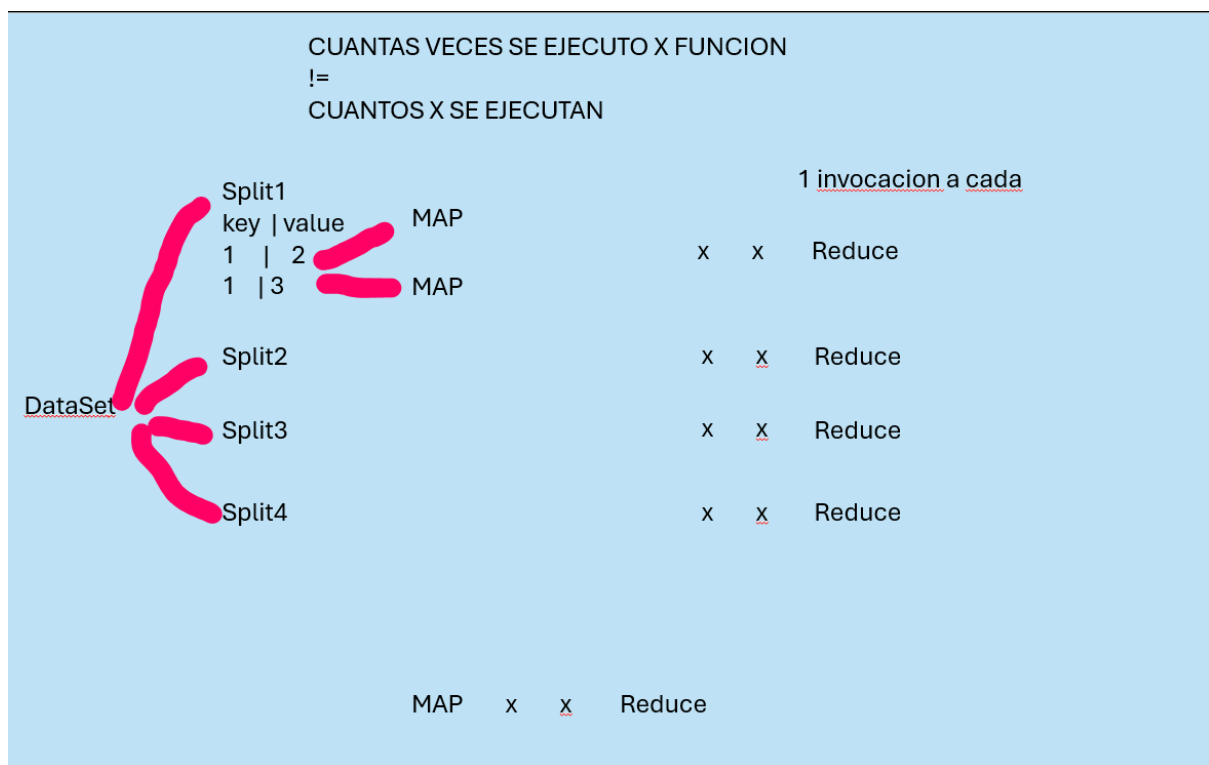
```
def map(k1, v1, context):
    context.write(v1, k1)

def reduce(k2, v2, context):
    n = 0
    for v in v2:
        n = n + 1
    context.write(v, n)
```

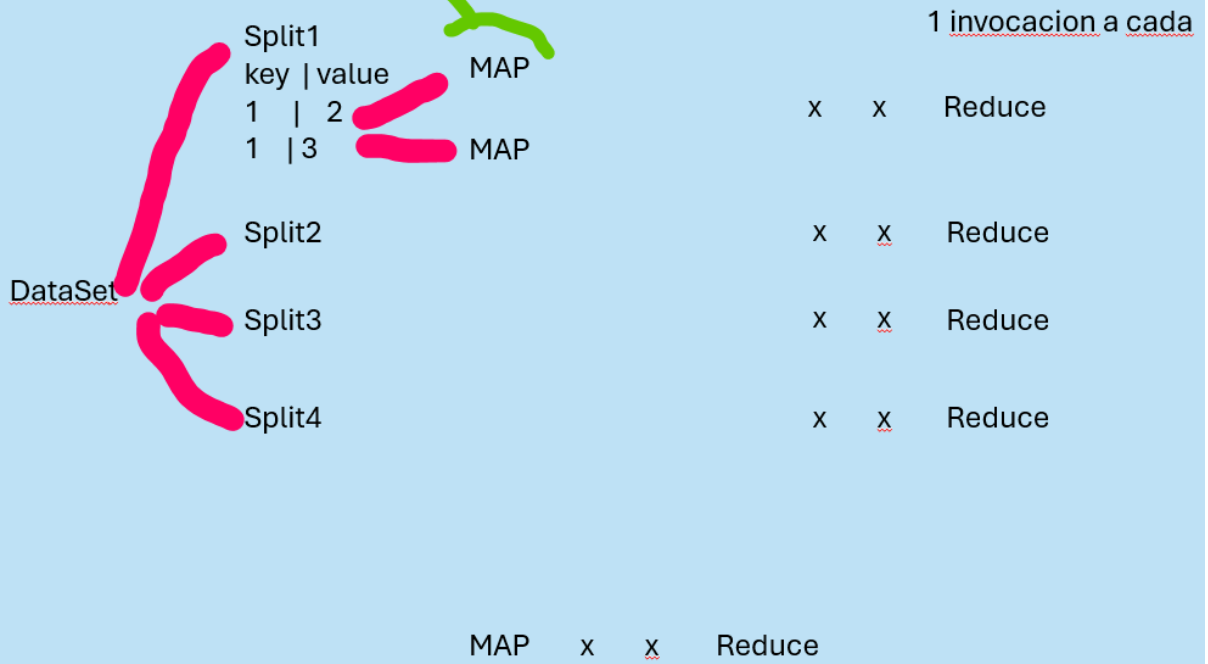
16.. 1 x cada fila del split
 12
 4
 12
 cuenta la cantidad de veces que aparece cada value

=====

“RESUMEN PARA LA PRACTICA”



CANT DE LLAMADOS A LA FUNCION
MAP



Cuando tenemos Splits

Cantidad de llamados a la función map

1 x cada fila de cada Split

Cantidad de mappers se ejecutan

1 x cada Nodo en el mayor de los casos

Cantidad de llamados a la función reduce

1 x cada split

Cantidad de reducers que se ejecutan

1 x cada Clave distinta que se obtenga del mapper

Salida del Job con 4 claves:

Clave (k_2)	Lista recibida (v_2)	Salida del <code>reduce</code>
1	[v1, v2, v3, v4]	(1, 4)
2	[v5, v6, v7, v8]	(2, 4)
3	[v9, v10, v11, v12]	(3, 4)
4	[v13, v14, v15, v16]	(4, 4)

Que datos recibe la función reduce?

Recibe una lista de Valores.. De cada nodo en el mayor de los casos..

Un Job = Una tarea analítica completa (Map + Reduce) sobre un dataset.

Procesamiento secuencial: Si hay dependencia de datos (el output de A es el input de B), van en estricto orden.

Procesamiento paralelo: Si son datos o procesos independientes, el cluster puede procesar varios jobs al mismo tiempo.

1) Dado el siguiente dataset:

Split 1		Split 2		Split 3		Split 4	
Key	Value	Key	Value	Key	Value	Key	Value
34	21	23	45	3	21	30	91
21	34	12	12	15	10	31	32
10	18	36	18	14	18	32	53
32	45	4	97	3	15	19	35

```
c) Job C
def map(k1, v1, context):
    if (v1 < 30):
        context.write(1, k1)
    else:
        context.write(2, k1)

def reduce(k2, v2, context):
    max = -1
    for v in v2:
        if (v > max):
            max = v
    context.write(k2, max)
```

- ¿Cuántas veces (invocaciones) se ejecuta la función map?
- ¿Cuántas veces (invocaciones) se ejecuta la función reduce?
- ¿Cuántos mappers se ejecutan?
- ¿Cuántos reducers se ejecutan?
- ¿Qué datos recibe cada función reduce?
- ¿Cuál es la salida de cada job?

Aca preguntar...

cuando tenemos ifs, la cantidad de reducees que se ejecutan es la cantidad de `context.write(valorDiferente, k1)` ?

o si o si hay que analizar si por lo menos entro 1 vez en el if y else para saber que tuvimos 2 reducees que se ejecutaron